

김지은

Frontend Engineer

기능 구현의 끝은 화면 완성이 아니라, 예외 상황에서도 흐름이 이어지는 구조라고 생각합니다.

React와 TypeScript 기반으로 권한 기반 화면 제어, 폼 검증, 공통 에러 처리, 실시간 로그 모니터링 등 서비스 운영 중 자주 마주치는 문제를 다뤘습니다. Zod 스키마 기반으로 검증 로직을 중앙화해 필수 검증 누락을 줄이고, SSE 로그 모니터링에서는 화면 전환과 토큰 만료 상황에서도 연결 흐름이 안정적으로 관리되도록 구현했습니다. 문제가 발생했을 때는 재현 조건을 좁혀 원인을 추적하고, 해결 과정을 문서화해 팀이 같은 문제를 반복해서 겪지 않도록 공유합니다. 단순히 기능을 빠르게 만드는 것보다, 운영 중에도 유지보수 가능하고 예외에 강한 화면 구조를 만드는 데 집중합니다.

jisilver.kim@gmail.com · 010-3724-9688

경력

(주)씨에이웨이브

2024.03 ~ 재직 중 (2년 3개월) · 개발팀 주임

클라우드/B2B 업무 솔루션 프론트엔드 개발

기술 스택

HTML / CSS

JavaScript

TypeScript

React

Zustand

React Query

React Hook Form

Zod

Tailwind CSS

Vitest

React Testing Library

프로젝트

제조 AI 데이터 통합·중계 플랫폼

2025.05 ~ 2026.03 · (주)씨에이웨이브

제조 데이터를 수집·연계 및 관리하고 데이터 탐색 및 검증 기능을 제공하는 데이터 허브 플랫폼 프로젝트

[다운로드 없이 확인·탐색 가능한 다중 포맷 데이터 미리보기 개발]

CSV, JSON, 이미지, 영상 등 5종 이상의 데이터 포맷을 플랫폼 내부에서 확인할 수 있는 자체 미리보기 컴포넌트를 개발했습니다.

01 문제 인식

- 데이터 확인·탐색 단계에서 파일을 다운로드해 별도 도구로 열어보는 과정이 반복되어 탐색 흐름이 끊겼음

- 긴 텍스트/로그 파일을 그대로 렌더링할 경우 초기 렌더링과 스크롤 성능이 저하될 수 있었음

02 접근 방식

- 포맷별 렌더링 전략을 분리하고 공통 미리보기 인터페이스 안에서 처리하도록 컴포넌트 구조 설계
- 텍스트성 데이터는 파일 길이에 따라 일반 렌더링과 **react-window 기반 가상 스크롤 렌더링**을 분기하여 화면 노출 영역 중심으로 렌더링

03 결과

사용자가 다운로드 없이 데이터 내용을 즉시 확인할 수 있어 플랫폼 안에서 수집-탐색 흐름을 이어갈 수 있게 했습니다.

[SSE 기반 실시간 로그 모니터링 안정화]

분석 서버 상세 화면에서 작업 로그를 SSE로 스트리밍하고, 연결 상태와 최근 로그 라인 수를 제어할 수 있는 모니터링 뷰어를 개발했습니다.

01 문제 인식

- 분석 작업은 장시간 실행될 수 있어 사용자가 진행 상태와 실패 원인을 실시간으로 확인할 수 있어야 했음
- 로그 스트림은 화면 전환, 분석 서버 변경, 토큰 만료 상황에서 기존 연결이 남거나 중복 연결될 위험이 있었음

02 접근 방식

- 네이티브 `EventSource` 대신 `@microsoft/fetch-event-source` 를 사용해 `Authorization` 헤더가 포함된 SSE 스트림 구성
- `AbortController` 로 기존 스트림을 정리하여 **서버 변경 및 컴포넌트 언마운트 시 불필요한 연결이 남지 않도록** 처리
- 401 응답 시 토큰 재발급을 시도하고, 연결 상태를 화면에 표시해 사용자가 현재 모니터링 가능 여부를 확인할 수 있도록 구현

03 결과

사용자가 별도 서버 접근 없이 플랫폼 안에서 분석 작업 로그를 확인하고, 화면 전환·서버 변경 시 기존 스트림이 남아 중복 연결될 가능성을 줄였습니다.

[테스트 환경 세팅 및 AI 기반 개발 워크플로우 구축]

Vitest 및 React Testing Library 기반 테스트 환경을 구축하고, 주요 UI와 핵심 비즈니스 로직을 검증하는 테스트 작성 흐름을 마련했습니다.

01 문제 인식

- 통합된 프론트엔드 테스트 환경이 부재하여, 리팩토링이나 신규 기능 추가 시 기존 로직의 안정성을 담보하기 어려웠음
- "이 함수나 컴포넌트가 제대로 동작하는가?"를 매번 수동으로 확인하면서 시간 및 심리적 비용이 증가했음

02 접근 방식

- 프로젝트의 Vite 기반 구조와 테스트 실행 속도, React Testing Library 호환성을 기준으로 테스트 도구를 비교
- 테스트 적합성 판단, 시나리오 설계, 모킹 기준, 실행 검증 절차를 AI 에이전트용 커스텀 스킬로 정리하여 테스트 코드 작성 품질과 일관성을 높임

비교 항목	Jest	Vitest (선택)
Vite 통합	babel-jest 변환 필요, 별도 설정 복잡	Vite 설정 그대로 공유 (vite.config.ts 에 test 블록 추가)
실행 속도	상대적으로 느림	ESM 네이티브 지원으로 빠름
HMR 연동	별도 구성 필요	vitest --watch 로 변경 파일만 즉시 재실행
호환성	jest-dom, RTL 완전 호환	jest-dom, RTL 동일하게 호환 (@testing-library/jest-dom)

이미 Vite 기반 프로젝트였기 때문에, 별도의 번들러 설정 없이 vite.config.ts 에 test 블록만 추가하면 되는 Vitest가 도입 비용이 가장 낮았습니다.

03 결과

지표	수치
테스트 대상	핵심 유틸 함수, Zod 스키마, 커스텀 훅 및 복잡한 UI 컴포넌트
테스트 케이스	팀 전체 430여 개 규모의 테스트 케이스 기반 구축에 기여
실행 속도	전체 테스트 약 2~3초 이내 완료 (Vitest 병렬 실행)
커버리지	주요 비즈니스 로직 기준 테스트 커버리지 80% 이상 달성

배포 전 주요 로직을 자동 검증할 수 있는 기반을 마련했습니다.

[사용자 권한 기반 화면 렌더링 제어]

전역 사용자 역할 상태를 기반으로 관리자 화면의 메뉴, 세부 컴포넌트, 액션 노출 조건을 제어하는 렌더링 로직을 구현했습니다.

01 문제 인식

- 관리자 계정으로 로그인했음에도 간헐적으로 관리자 전용 메뉴가 노출되지 않거나, 토큰 재발급 시 권한이 GUEST로 초기화되는 버그 발생

02 원인 분석

- 유저의 권한 데이터를 서버에서 모두 받아오기 전에 화면 전환이 먼저 일어나면서 빈 상태를 읽어 기본 권한인 GUEST가 할당되는 비동기 데이터 처리 순서 꼬임이 근본 원인

- 토큰 재발급 로직에도 동일한 인증 플로우가 엮여 있어 갱신 시 기존 권한마저 지워지는 부작용 발생

03 해결 과정

- 도착 순서에 의존하던 로직을 폐기하고, **실제 관리자 전용 API 호출 성공 여부를 기준으로 권한 상태를 판별**하는 확실한 방식으로 변경
- 토큰 재발급 프로세스와 권한 부여 로직을 완전히 분리하여, 토큰 갱신 시 권한 데이터가 간섭받지 않도록 독립적인 구조로 개선

04 결과

초기 진입, 토큰 만료 직후, 계정 간 전환 등 재현 시나리오에서 권한 상태가 의도치 않게 초기화되지 않도록 개선했습니다.

[트러블슈팅] 비동기 환경에서 실행 순서를 암묵적으로 가정하는 설계가 얼마나 위험한지 체감했습니다. 이후 상태 변경 로직을 설계할 때 항상 '초기 상태 첫 진입' 시나리오와 비동기 순서를 의심하고 검증하는 습관을 갖게 되었습니다.

클라우드 빌링 사용자 포털 및 관리자 콘솔 개선

2024.03 ~ 현재 · (주)씨에이웨이브

사용자 포털과 별도 관리자 콘솔의 운영 화면을 함께 개선하며, 자원·권한·역할 관리와 정산 폼 입력 흐름을 정리했습니다.

[공통 API 에러 안내 흐름 개선]

백엔드 예외 상태와 에러 코드를 사용자 안내 메시지로 변환하는 공통 에러 처리 혹은 구현했습니다.

01 문제 인식

- API 실패 상황을 화면별로 개별 처리하면 동일한 예외도 서로 다른 메시지로 안내될 수 있었음
- 백엔드 에러 코드가 늘어나면서 각 화면에서 메시지 매핑을 직접 관리할 경우 중복과 누락 가능성이 커졌음

02 접근 방식

- 에러 코드별 사용자 안내 문구와 snackbar variant를 매핑하는 `useErrorMessage` 혹은 구현
- Not Found 계열 예외는 안내 메시지와 분리해 전용 라우팅으로 처리하고, 화면별 기본 메시지 override가 가능하도록 구성

03 결과

여러 운영 화면에서 동일한 에러 안내 흐름을 재사용할 수 있게 하고, 사용자가 실패 상황을 일관된 메시지로 인지할 수 있도록 개선했습니다.

[별도 관리자 콘솔 기반 역할·권한·자원 관리 화면 구축]

사용자 포털의 역할별 접근 제어에 필요한 자원, 권한, 서비스 역할, 사용자 배정을 별도 관리자 콘솔에서 관리할 수 있도록 구현했습니다.

01 문제 인식

- 사용자 역할에 따라 접근 가능한 메뉴와 기능이 달라, 운영자가 권한 구성을 직접 관리할 화면이 필요했음
- 기존에는 권한 변경이 화면에서 즉시 처리되지 않아 개발자나 백엔드 담당자에게 요청해야 했고, 운영자가 변경 이력과 현재 구성을 한눈에 확인하기 어려웠음

02 접근 방식

권한·자원·역할 관리 화면을 구성하고, 서비스 역할에 권한과 사용자를 배정하거나 회수할 수 있는 모달 흐름을 구현했습니다.

03 결과

관리자 콘솔에서 역할과 권한 구성을 관리할 수 있어 사용자 포털의 역할 기반 접근 운영이 가능해졌습니다.

[Zod 및 React Hook Form 기반 복잡한 정산 폼 개선]

수십 개 입력 필드와 정산 조건을 가진 사용자 포털의 정산 폼에서 검증 로직을 스키마로 분리하고, 탭 구조와 폼 상태 유지 흐름을 함께 개선했습니다.

01 문제 인식

- 기존 폼 검증 로직이 각 UI 컴포넌트의 `register` 옵션과 커스텀 함수 조합에 흩어져 있어, 필드가 늘어날수록 필수 검증 누락 가능성이 커졌음
- 정산 정보, 과금 조건, 사업자 정보 등 입력 항목이 한 화면에 길게 이어져 사용자가 현재 작성 맥락을 파악하기 어려웠음

02 접근 방식

- `React Hook Form`의 `resolver`에 `Zod` 스키마를 연동하여 필수값, 형식, 조건부 입력값 검증 기준을 스키마 단위로 관리
- 방대한 단일 폼 화면을 사용자 업무 흐름에 맞춘 탭 구조로 분리하고, 탭 전환 중에도 입력 상태가 유지되도록 폼 상태 관리 구조 개선

03 결과

- 검증 기준을 스키마 단위로 확인할 수 있게 되어 필수 검증 누락 가능성을 줄이고 폼 로직의 유지보수성을 높였습니다.
- 긴 입력 화면을 탭 단위로 나누어 사용자의 입력 피로도를 낮추고, 정산 정보 입력 흐름을 더 안정적으로 이어갈 수 있게 했습니다.